

goorm

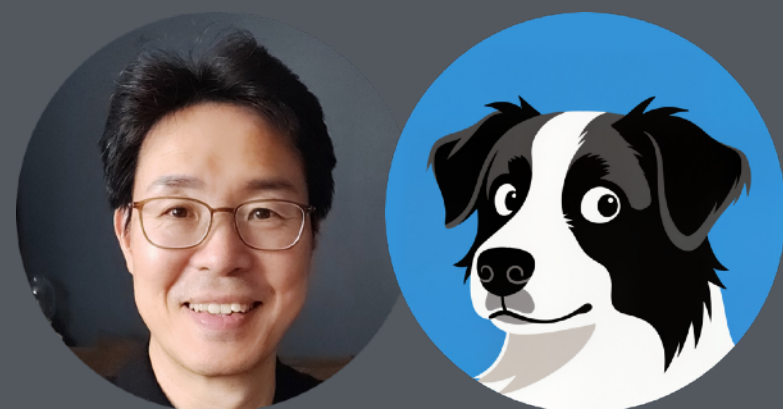


OPEN

AI Fit-T 프로젝트 : 바이트코딩 (25.09.05~10.02)

Cursor, Code, v0 바이트코딩 20제 마스터

Cursor, Code, v0 바이브코딩 20제 마스터



JUN (동준상)

- 넥스트플랫폼 대표 (naebon@naver.com), AX 저술/강연/컨설팅
- KDT 해커톤 (노동부), SW 융합 해커톤 (과기부) 바이브코딩 멘토링
- '데이터 마이닝 개념과 기술 4판', 'AWS SAA 시리즈' 등 20여권 저술
- 구름, KPC 한국생산성본부, SDS 멀티캠퍼스, GSC 글로벌 소프트웨어 캠퍼스 인공지능 및 클라우드 강연 (현대오토에버, KIA, 하나금융 강연)
- 예스24 '이달의 개발자', 삼성SDS SCP 인증자격 출제위원
- 저서: 데이터 마이닝 개념과 기법 4판, 머신러닝 인 자바, 마스터링 자바 머신러닝, AWS 스터디가이드 3, 4판 외 20여권 저술 및 번역

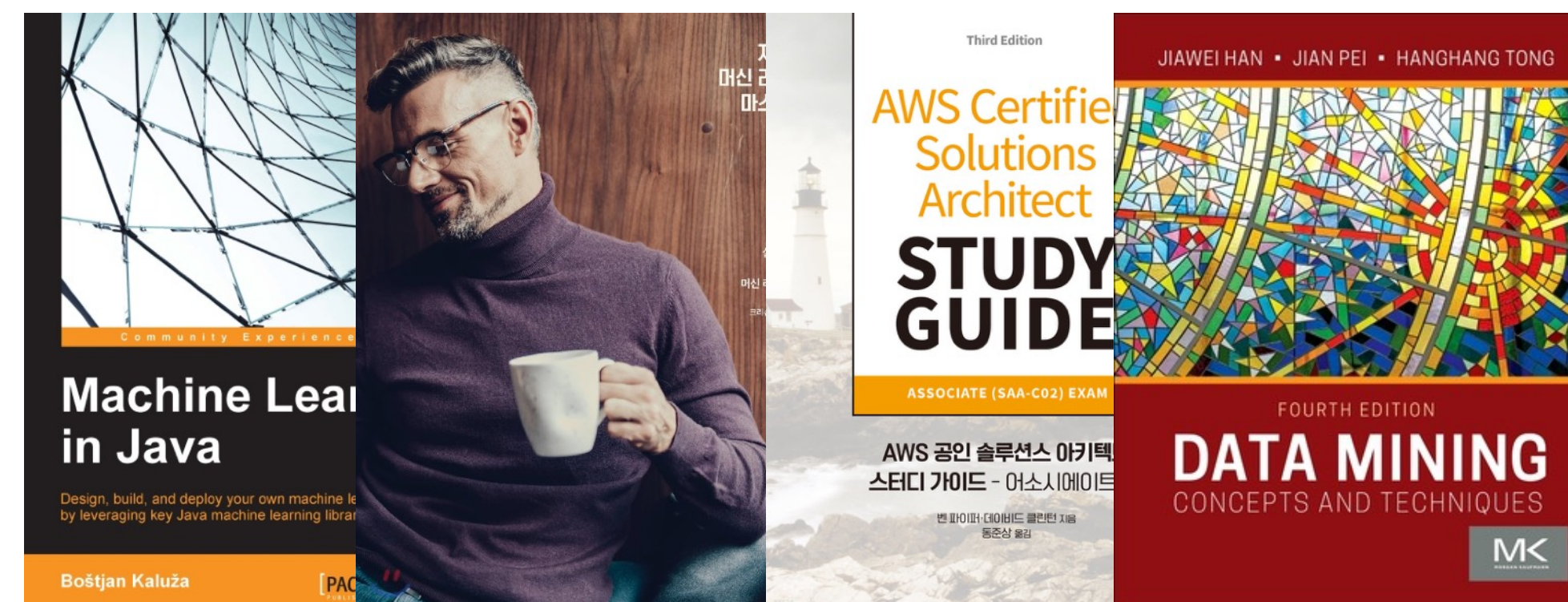
+

강연기관

SAMSUNG SDS



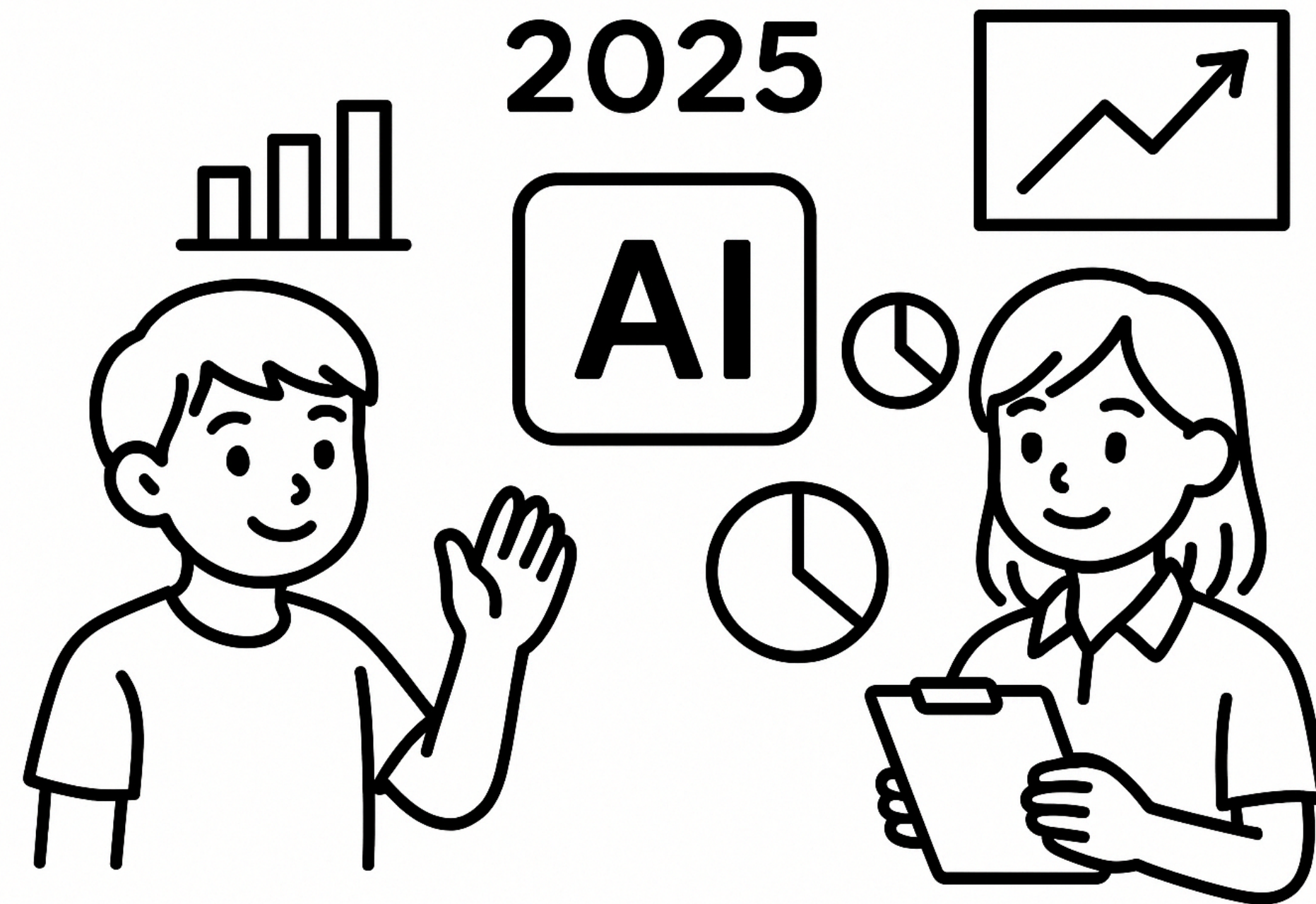
저서/번역서



<https://nextplatform.net/동준상-넥스트플랫폼-프로필/>

목차

1. 바이트코딩 시작하기
2. 입문자 바이트코딩 도구
3. 모범적인 바이트코딩 방식
4. 바이트코딩 입문 프로젝트
5. 바이트코딩 응용 프로젝트
6. 바이트코딩 점프 스타트
7. 바이트코딩 실전 프로젝트
8. 바이트코딩의 한계영역



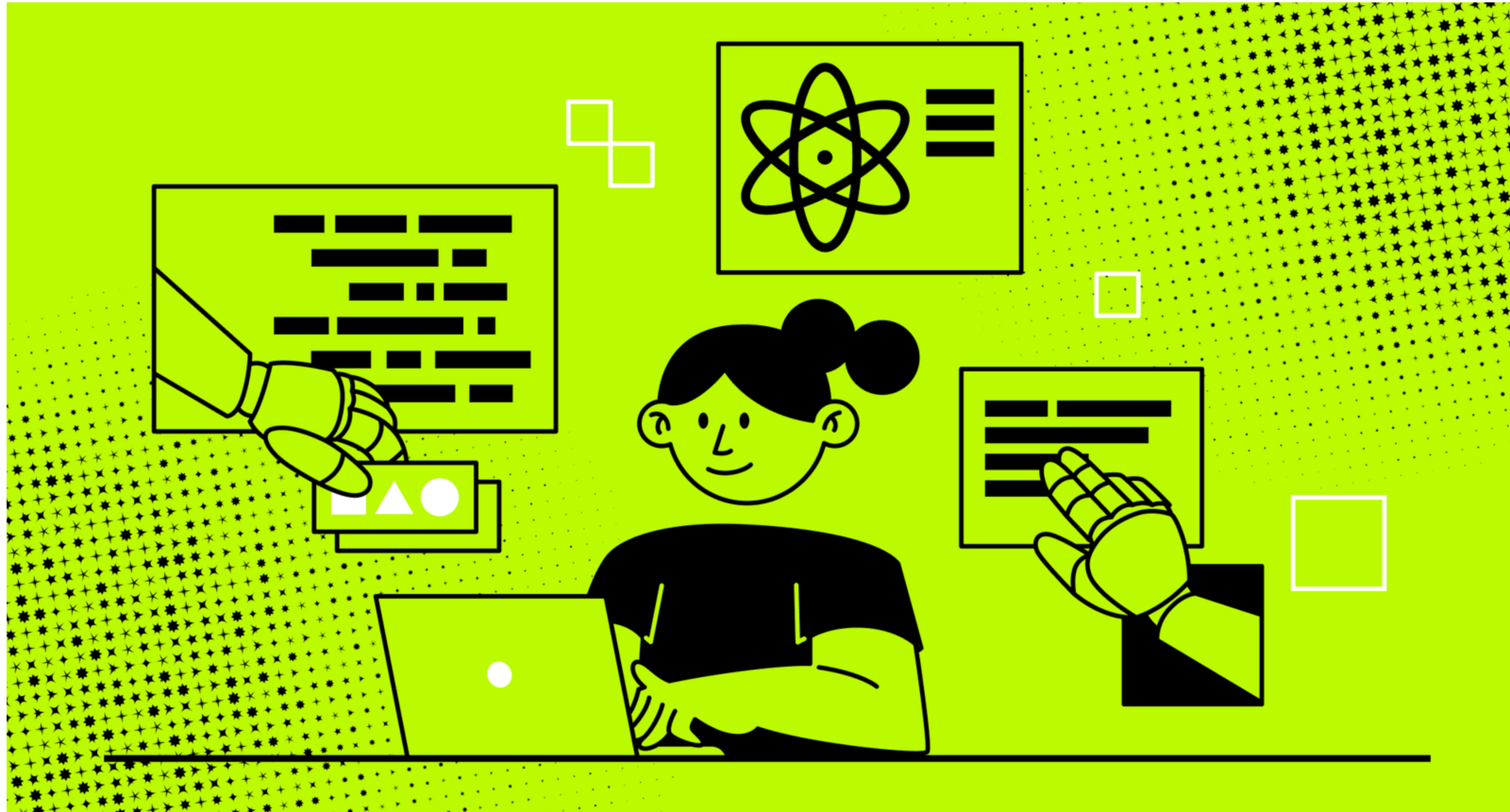
VibeCoding Master

바이브코딩 시작하기

(Getting Started VibeCoding)



바이트코딩 세계로 오신 것을 환영합니다.

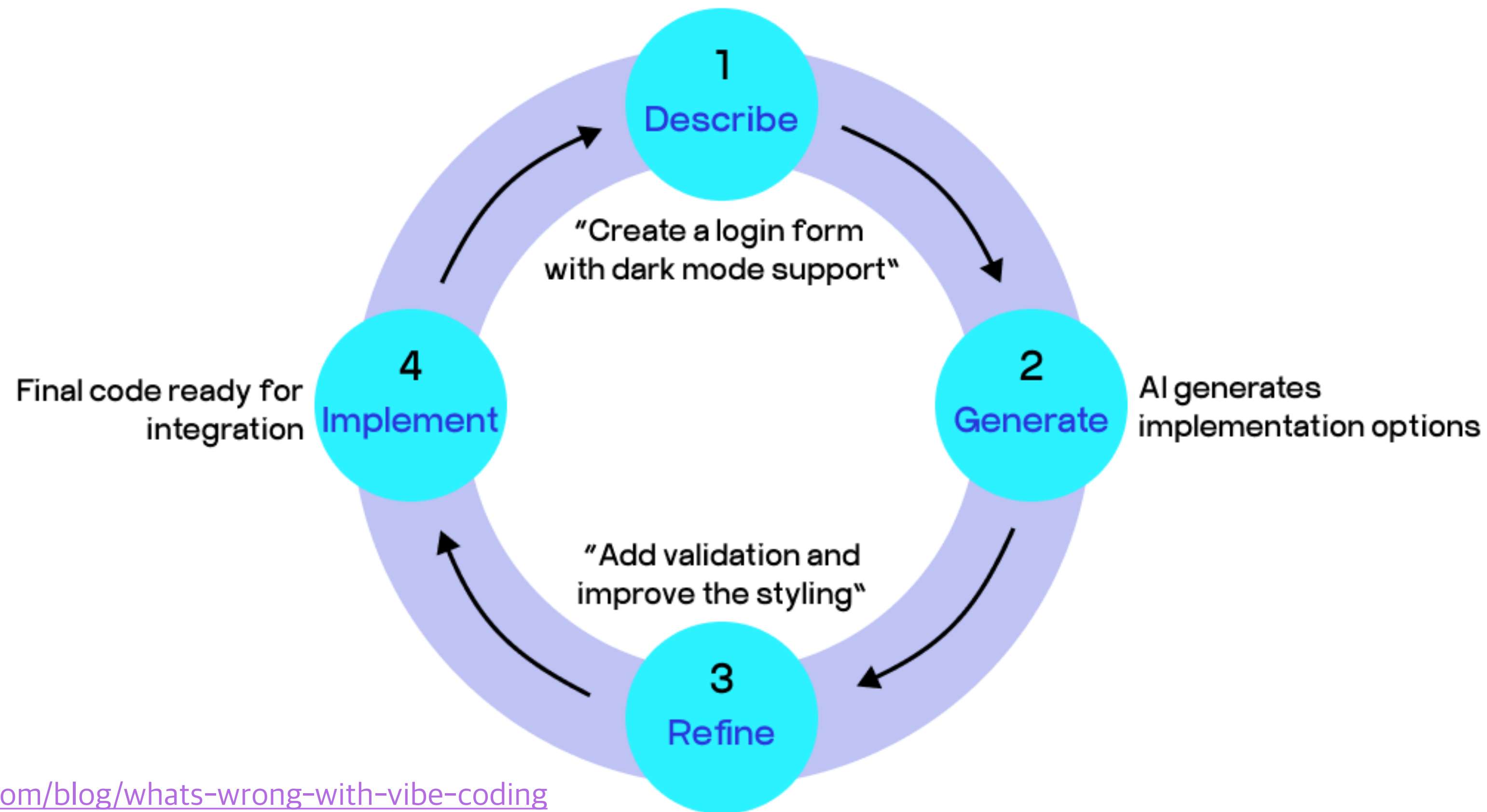


시작하기: 바이트코딩이란?

- AI와 함께하는 실시간 프로그래밍 방식
- 전통적 코딩 지식 → AI 활용 역량으로 재분배
- 문맥 관리, 코드 검증, 반복 개선 중심
- 목표: 기획→개발→배포의 전 과정을 AI와 협업

바이트코딩 4대 프로세스

The vibe coding workflow



바이트코딩과 기존 코딩 방식 비교

Aspect	Vibe coding	Traditional coding
Required skills	Communication ability, clear explanation of needs	Programming languages, algorithms, debugging
Development speed	Rapid prototyping, results in hours or days	Slower development cycle with thorough planning
Code understanding	Limited knowledge of how the code works	Complete understanding of implementation details
Maintenance	Challenging to modify without AI assistance	Easier to adjust, debug, and optimize
Learning investment	Minimal technical learning needed	Months/years mastering programming fundamentals
Project scale	Better for smaller projects and prototypes	More suitable for complex, enterprise applications

바이트코딩 기획1: 목표 설정

- 서비스 목적과 핵심 기능 정의
- 기술 스택, 인증, 수익화 전략 구상
- 초기 다이어그램/워크플로우 작성
- AI 제안 → 의도 수정/구체화 가능

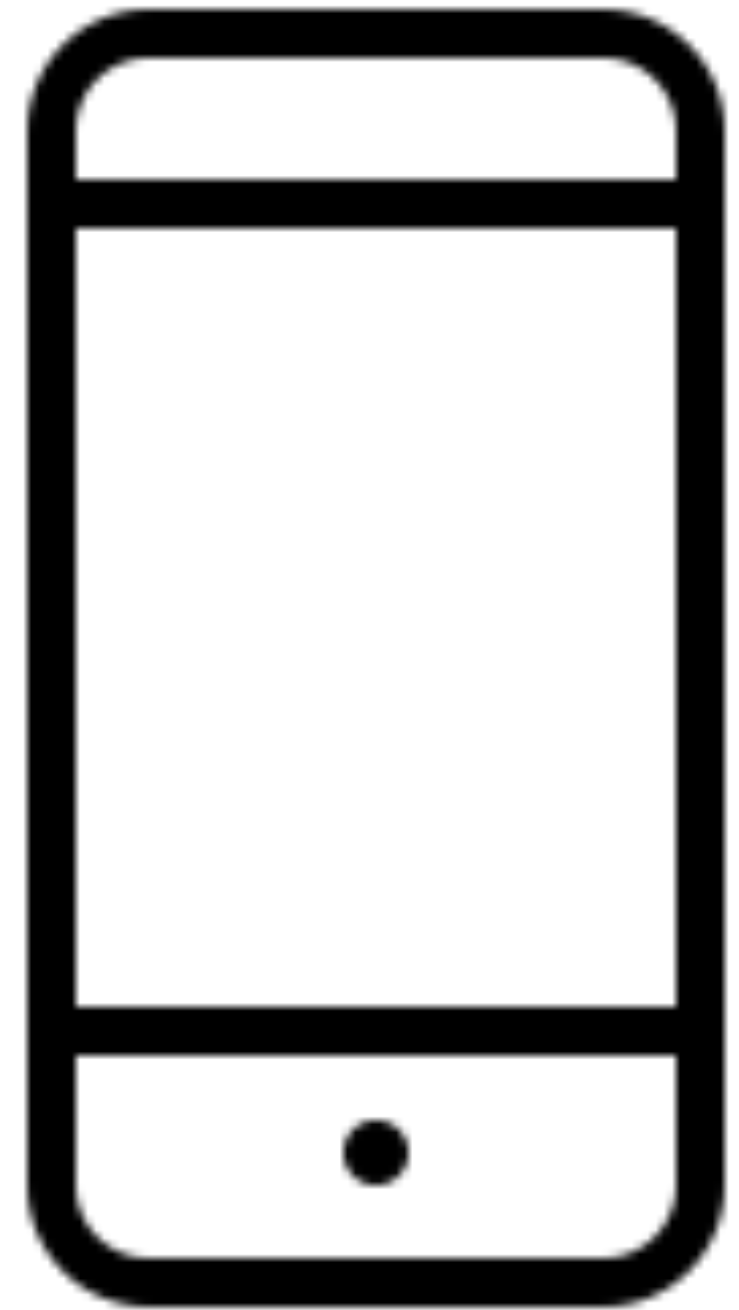
기획 예시0: GPT에 요청

영어 문화권 사용자를 대상으로 iOS용 한국어 학습 앱을 만들 계획이야.
한국어 학습 콘텐츠는 30초에서 90초 길이의 유튜브 영상 클립을 URL로 연결할 수 있게 해줘.
위 작업을 위한 모범적인 바이트코딩 업무 흐름을 제안해주겠어?



기획 예시1: 목표, 전략

- 앱 콘셉트: 영어권 사용자 대상 한국어 학습
(30-90초 YouTube 클립 활용)
- 핵심 가치: 짧고 몰입도 높은 학습 → 발음·표현
- MVP 범위: 영상 재생, 이중 자막, 표현 카드
- 바이브코딩 원칙: 아이디어 → 프롬프트 → 코드 → 실행 → 검증 (하루 단위 루프)



기획 예시2: 시스템, 데이터 스키마

- iOS: SwiftUI + WKWebView(iFrame)
- 백엔드: Supabase (콘텐츠/진행 저장)
- 분석: Firebase Analytics
- clips: 유튜브 URL, 제목, 주제, 난이도
- cards: 표현/의미/로마자/태그
- quizzes: MCQ/번역/순서 배열



기획 예시3: 콘텐츠 & UX 설계

- 콘텐츠 파이프라인: 영상 큐레이션 (저작권)
- 자막 확보 → 카드 & 퀴즈 자동 생성 (LLM)
- UX 요소: KO/EN 자막 + 하이라이트 싱크
- 카드 스와이프 학습 + 퀴즈 3문항
- 프롬프트 예시: “자막에서 표현 5개 추출
→ {ko, romanization, en} JSON 카드 생성”



바이트코딩 기획2: 도구, 모델 선택

- 프로젝트 성격에 맞는 IDE 선택 (Cursor 등)
- 모델별 장단점 비교 (속도/품질/비용)
- 다양한 AI 모델 혼합 사용 (Claude, Gemini 등)
- 실제 사용 사례: Cursor 선택 이유

바이트코딩 기획3: 프롬프트 준비

- Code Guide 등 메타 도구 활용
- 요구사항을 문서화하여 환각 최소화
- 시스템 지침 수정: 언어·프레임워크 지정
- 프롬프트 사전 처리로 AI 품질 향상

바이트코딩 개발1: 반복적 사이클

- (1) 목표 설정 → (2) 코드 생성 요청
- (3) 코드 검토 → (4) 수락/거부
- (5) 테스트 실행 → (6) 문제 식별
- (7) 프롬프트 개선/수동 디버깅
- 이 과정을 반복하며 완성도 상승

바이트코딩 개발2: 프롬프트 전략 ①

- 문맥 초기화: 새 창/탭으로 집중 관리
- 함수·API 이름 등 구체적 참조 활용
- 구조화된 지시: 번호 매기기/조건 제시
- JSON 예시, 오류 메시지 제공 → 품질 향상

바이브코딩 개발3: 프롬프트 전략 ②

- 외부 문서 URL 첨부 → 신뢰도 향상
- 프롬프트 범위 제한(“Stripe는 나중에”)
- 고수준→세부 정보로 점진적 상세화
- 복잡도 단계별로 세분화된 요청

바이트코딩 개발4: 코드 검토

- AI 코드 전체 구조(gestalt) 빠른 파악
- 시각적 diff로 변경 사항 즉시 확인
- 주요 함수 호출·주석·API 사용 확인
- 실행 전 코드 의도와 동작 예측 가능

바이트코딩 개발5: 디버깅 전략

- 하이브리드: AI + 수동 병행
- 브라우저 콘솔/네트워크 탭 활용
- 오류 메시지 AI에 직접 전달 후 수정 요청
- 모델 전환(Claude ↔ GPT ↔ Gemini) 활용

바이트코딩 개발6: 수동 개입

- 간단 수정 → 직접 편집이 더 효율적
- 사소한 UI 요소 정제·삭제 시 직접 변경
- 복잡한 버그 진단 → 전통적 디버깅 의존
- 통제력 유지 차원에서 수동 검토 선호

바이트코딩 배포1: 품질 관리

- 코드 복잡도·유지보수성 최종 점검
- 설계 표준 및 의도 적합성 확인
- 실제 운영 환경 통합 여부 확인
- PR 검토 후 배포 권장

바이브코딩 배포2: 문서화

- 프로젝트 계획·문서 최신화 필요
- 버전/라이브러리 정보 확인 & 수정
- 실제 코드와 문서의 불일치 보정
- AI 생성 문서 → 최종 수동 검증 필수

바이트코딩 체크리스트1: 전문성

- 전통적 코딩 지식은 여전히 중요
- AI 모델 한계·비용·성능 이해 필요
- 제품 관리 역량: 목표→기능 변환
- 새로운 역량: “주변 역량(Ambient Competence)”

바이브코딩 체크리스트2: 신뢰 관리

- 맹목적 수용 X → 지속적 검토 필요
- 존재하지 않는 API 제안 거부
- 개인 전문성에 따라 신뢰도 조절
- 신뢰 = 반복 검증을 통한 관계 구축

바이트코딩 체크리스트3: 문맥 관리

- 새 세션/탭으로 문맥 초기화
- 초반 프롬프트 → 프로젝트 방향 좌우
- 필요 시 과감히 재설정
- 문맥 모멘텀(Context Momentum) 관리

바이브코딩 체크리스트4: 프롬프트

- 고수준 + 상세 지시 혼합 활용
- 구조화된 지침/예시/외부 문서 포함
- 환각 줄이기 위해 범위 제한
- 점진적 구체화로 명확성 확보

바이브코딩 체크리스트5: 디버깅

- AI = 보조 도구, 전통 디버깅 병행
- 오류 메시지 분석·터미널 활용
- 복잡 버그 → 직접 가설·수정 지시
- AI에 “구체적 수정” 요청하기

바이브코딩 체크리스트6: 수동 개입

- 효율성: 사소한 변경은 직접
- 정제: AI 코드 보완은 수동 편집
- 복잡 버그: 직접 진단 후 AI에 실행 지시
- 개발자 통제력 유지의 핵심

바이브코딩 체크리스트7: 사회적 영향

- AI 사용 = 부정적 인식 가능성 존재
- 투명하게 AI 활용 과정 공유 필요
- 기술적 전문성·디버깅 능력 강조
- “AI를 잘 안내하는 역량” 시연

정리: 바이트코딩 실무 적용 가이드

- 기획→개발→배포: AI와의 협업 최적화
- 인간 전문성 = AI 활용 성공의 필수 요소
- 신뢰/문맥/프롬프트/수동 개입 → 4대 원칙
- 기획자의 지식, 의도, 경험을 담아 / AI의 폭 넓은 통섭 능력을 활용하여 / 사용자 친화적인 소프트웨어 기능을 구현

VibeCoding Master

입문자 추천 바이브코딩 도구

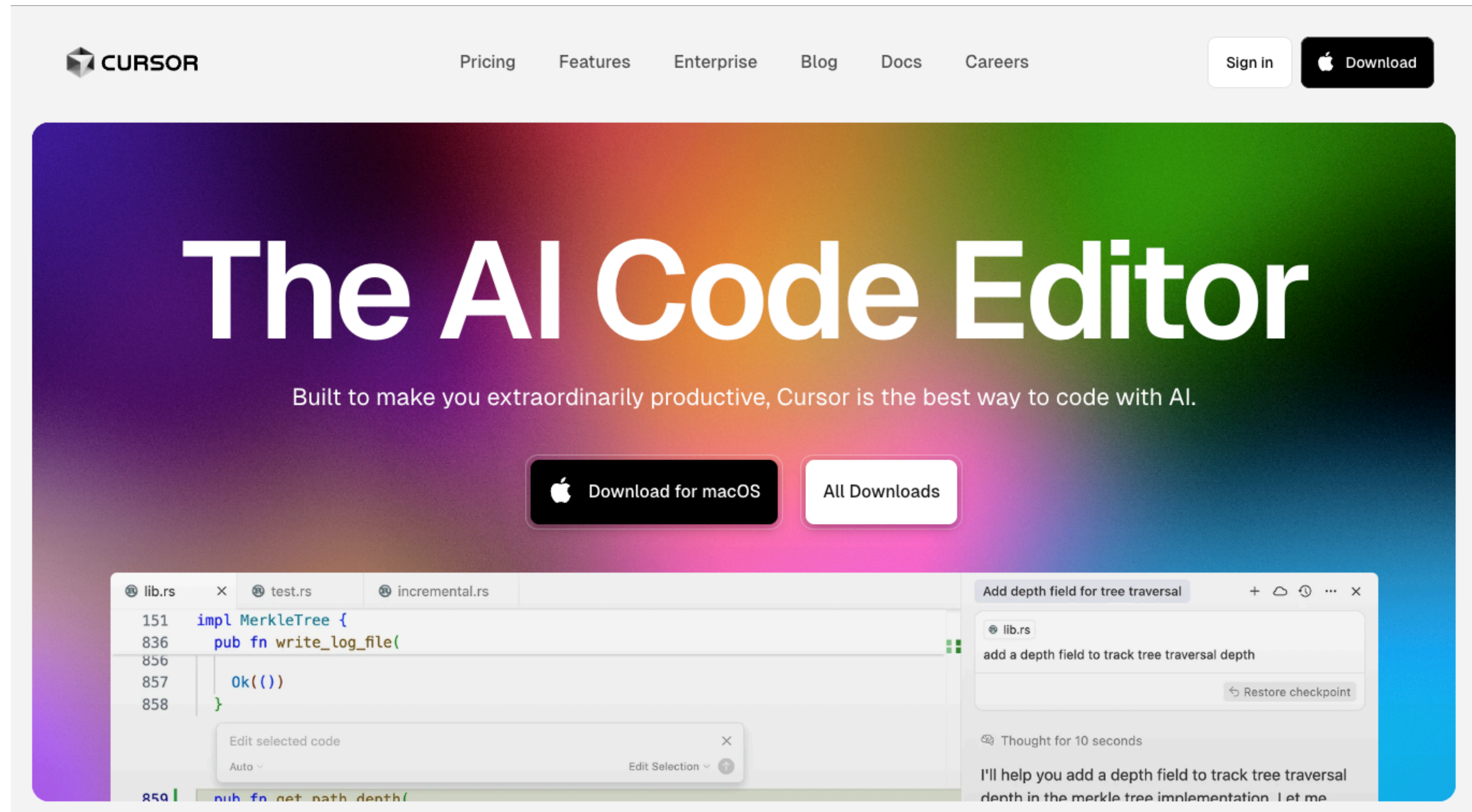
(Best Tools for VibeCoding Beginner)



1. Cursor

- VS Code 기반 IDE, AI 코드 작성·리팩토링·검색 최적화
- GPT/Claude/Gemini 모델 자유 연결
- 프로젝트 단위 System Prompt & Rule 관리
→ 바이트코딩 핵심 툴
- 웹/앱, 자동화, SaaS 템플릿까지 확장성 최고
- MCP로 외부 API·DB 연동

1. Cursor



1. Cursor



영어 ▾

Q 찾다...

₩ K

🌟 AI에게 물어보세요

로그인

다운로드 >

⚙️

선택 서류 비치

CLI

가이드

도구

🌐 웹사이트

📄 법정

🗣️ 지원하다

시작하기

환영

설치

빠른 시작

개념

핵심

꼬리표

대리인 >

백그라운드 에이전트

백그라운드 에이전트 API >

웹 및 모바일

인라인 편집 >

구현

시작하기

빠른 시작

5분 안에 커서를 시작하세요

이 빠른 시작에서는 커서의 핵심 기능을 사용하는 프로젝트를 안내합니다. 이 과정을 마치면 탭, 인라인 편집, 에이전트에 익숙해질 것입니다.

커서에서 프로젝트 열기

기존 프로젝트를 사용하거나 예제를 복제하세요.

복제 예제 프로젝트

기존 프로젝트 사용

1. git이 설치되어 있는지 확인하세요

2. 예제 프로젝트를 복제합니다.

```
git clone git@github.com:voxelize/voxelize.git && \  
cd voxelize && \  
cursor .
```

📄 🌟

☰ 이 페이지에서

커서에서 프로젝트 열기

탭으로 자동 완성

선택 항목 인라인 편집

에이전트와 채팅

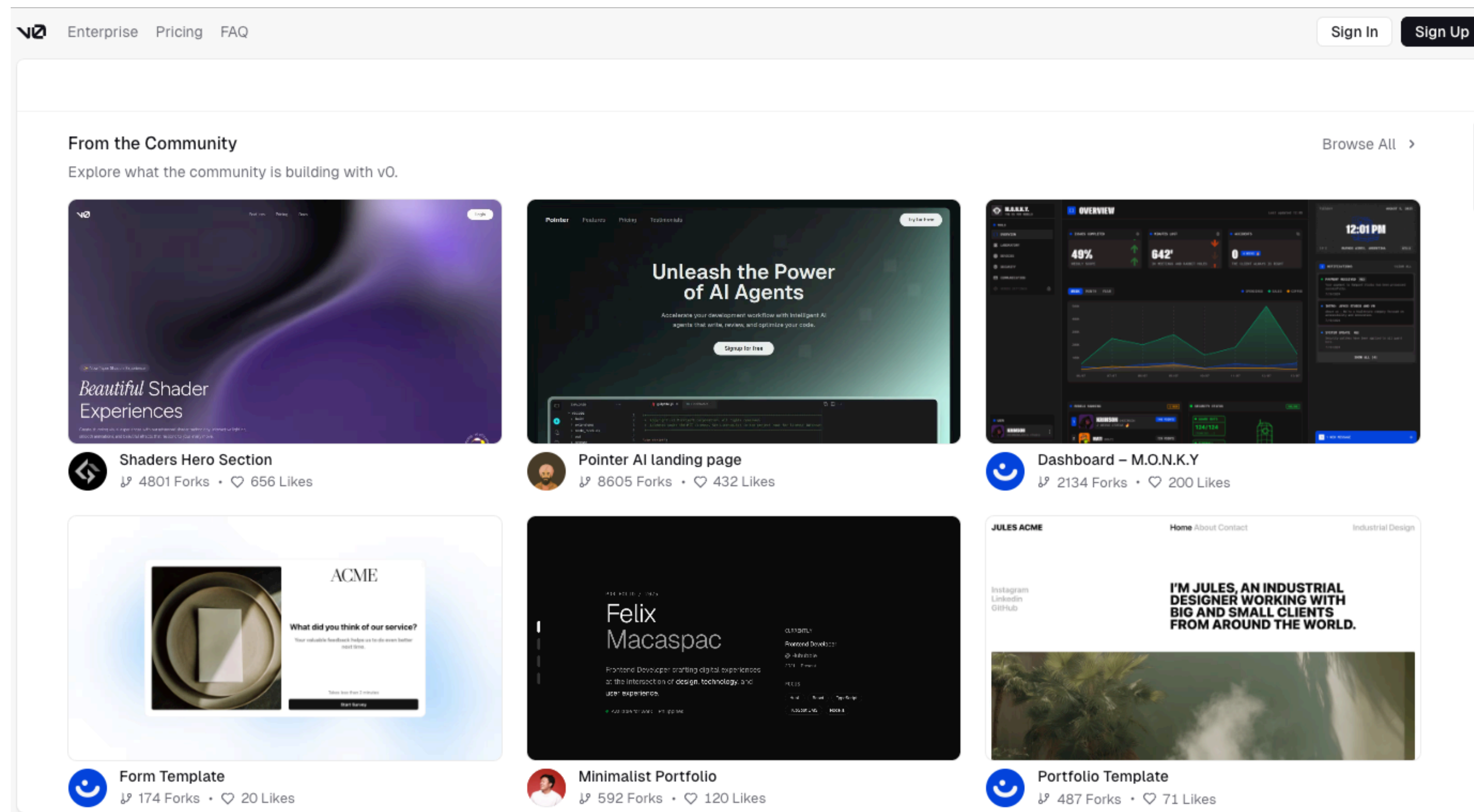
보너스

다음 단계

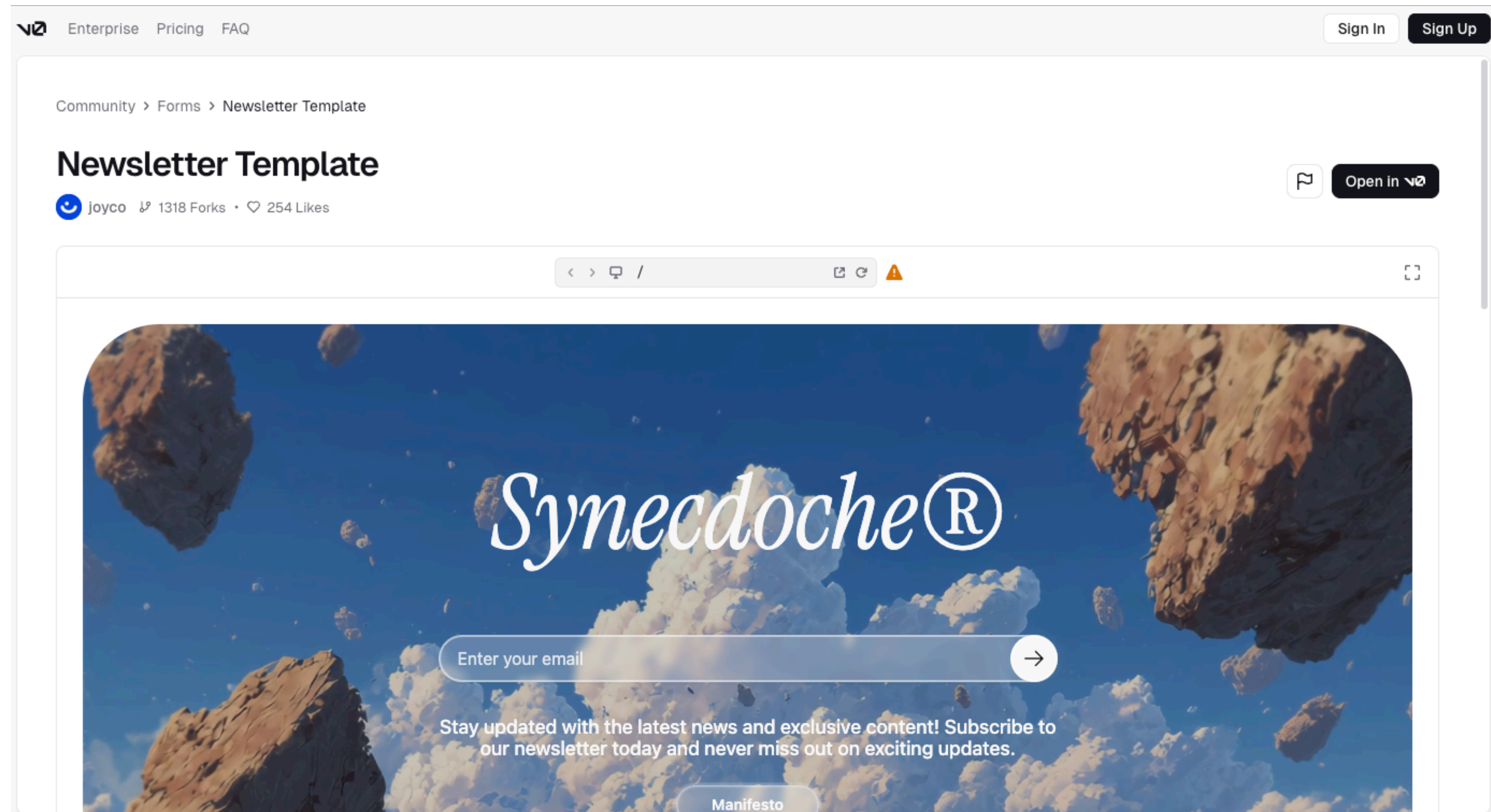
2. v0 (by Vercel)

- 프롬프트만으로 UI 컴포넌트를 코드로 생성하는 서비스
- React/Tailwind 기반 UI 자동 생성
- 디자인→코드→배포까지 빠른 루프 지원
- 입문자도 “프롬프트 한 줄”로 실행 가능한 웹페이지를 곧바로 배포

2. v0 (by Vercel)



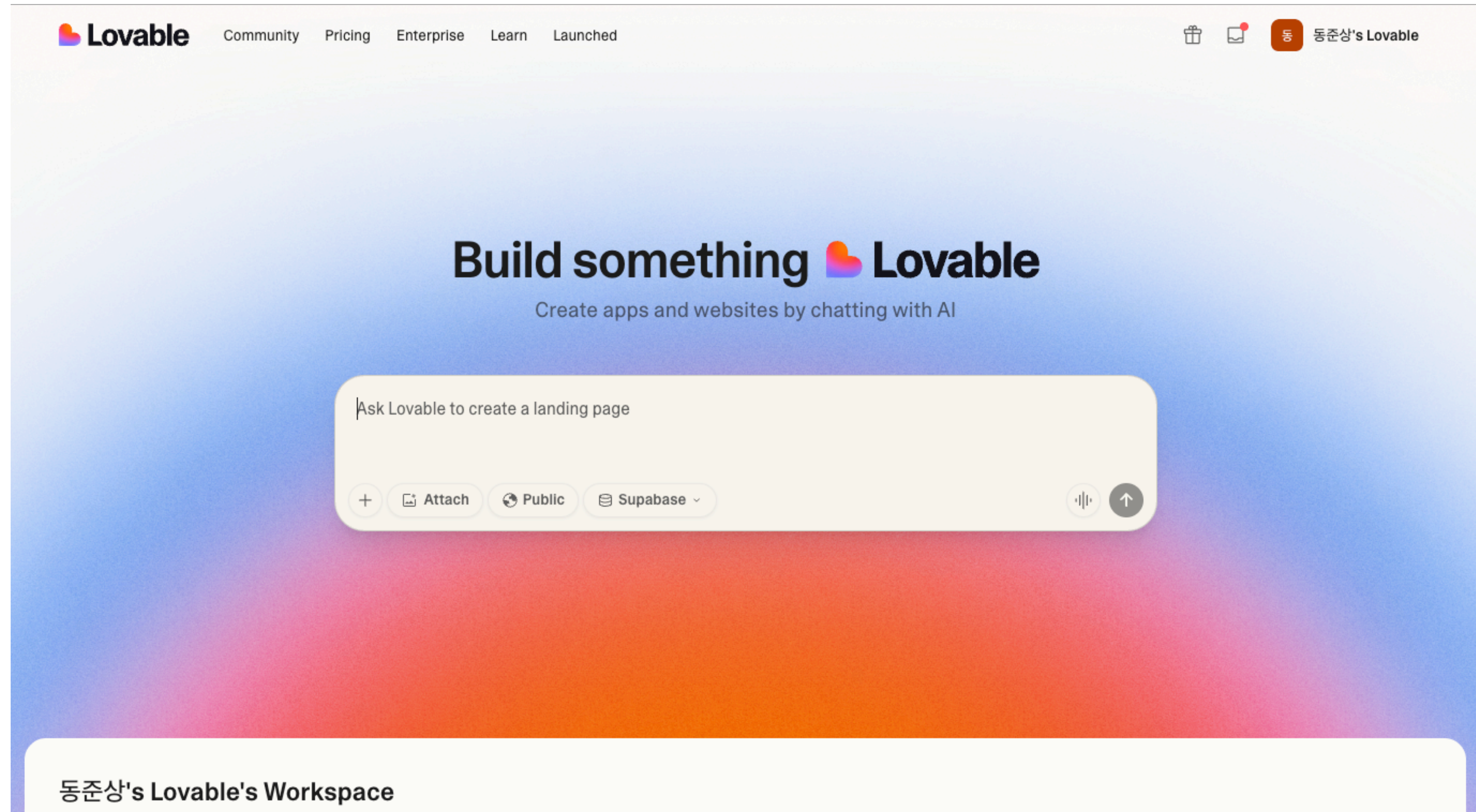
2. v0 (by Vercel)



3. Lovable

- 노코드 + AI 결합 웹앱 빌더
- 간단한 설명만으로 웹앱 자동 생성
- 구글 시트, Stripe 같은 외부 서비스 통합
- 비개발자도 SaaS 실험을 할 수 있는 최적의 도구

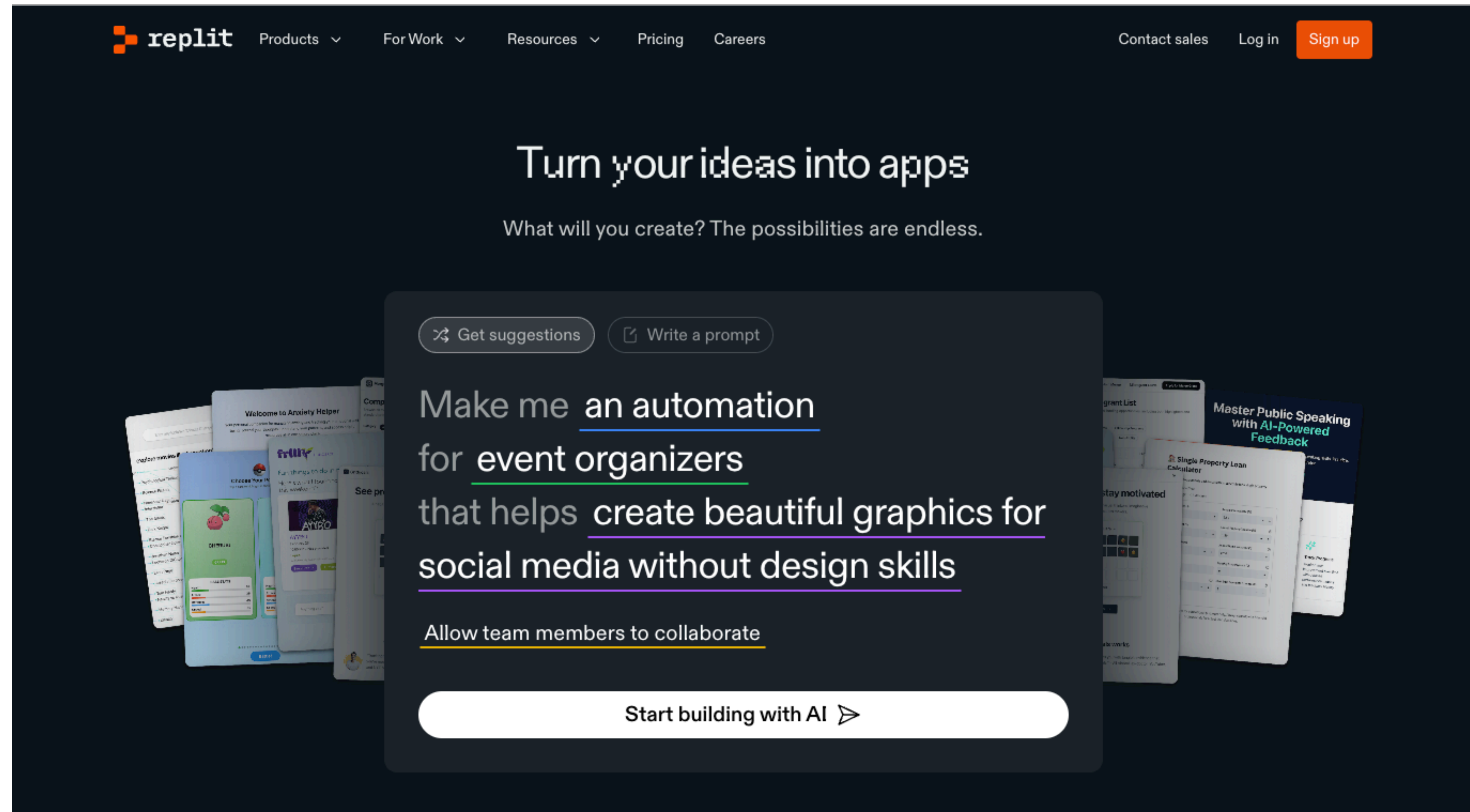
3. Lovable



4. Replit + Ghostwriter

- 온라인 IDE + AI 코드 어시스턴트
- 브라우저에서 협업 코딩, 멀티 언어 지원
- 배포까지 올인원 지원
- 학습자·팀 프로젝트에서 환경 세팅 없이 바로 실습 가능

4. Replit + Ghostwriter



The image shows the Replit website header and hero section. The header is dark blue with the Replit logo (an orange square with a white 'x' shape) and the word 'replit' in white. To the right of the logo are links for 'Products', 'For Work', 'Resources', 'Pricing', and 'Careers'. Further right are links for 'Contact sales', 'Log in', and a prominent orange 'Sign up' button. The hero section has a dark blue background with the text 'Turn your ideas into apps' in large white font, followed by the tagline 'What will you create? The possibilities are endless.' in a smaller white font. Below this is a dark grey rounded rectangle containing two buttons: 'Get suggestions' (with a magic wand icon) and 'Write a prompt' (with a document icon). The main text in the hero section is 'Make me an automation for event organizers that helps create beautiful graphics for social media without design skills'. Below this is the text 'Allow team members to collaborate' and a large white button that says 'Start building with AI' followed by a right-pointing arrow. On the left and right sides of the hero section, there are several overlapping images of various web applications created on Replit, including a 'Welcome to Anxiety Helper', a 'fruity' app, a 'Single Property Loan Calculator', and a 'Master Public Speaking with AI-Powered Feedback' app.

replit Products ▾ For Work ▾ Resources ▾ Pricing Careers

Contact sales Log in Sign up

Turn your ideas into apps

What will you create? The possibilities are endless.

Get suggestions Write a prompt

Make me an automation for event organizers that helps create beautiful graphics for social media without design skills

Allow team members to collaborate

Start building with AI ➤

5. Claude Code, Gemini Code

- AI 모델을 “코딩 동반자”로 쓰는 CLI 도구
- 터미널 기반 프롬프트 → 코드/스크립트/자동화 제안
- Cursor와 연계하면 강력한 듀오
- 바이트코딩 자동화 파이프라인에 적합 (예: 크롤러, 배치 작업, API 연결)

5. Claude Code

ANTHROPIC

Claude ▾API ▾Solutions ▾Research ▾Commitments ▾Learn ▾News

Try Claude

Your code's new collaborator

Unleash Claude's raw power directly in your terminal. Search million-line codebases instantly. Turn hours-long workflows into a single command. Your tools. Your workflow. Your codebase, evolving at thought speed.

Try Claude Code on Max

[See our pricing options](#)

A graphic of a terminal window with a dark background. Inside the terminal, the words "CLAUDE" and "CODE" are displayed in a large, orange, pixelated font. The letters are composed of small squares, giving it a retro, digital appearance. The terminal window has a standard macOS-style title bar with red, yellow, and green window control buttons in the top left corner.

5. Claude Code

ANTHROPIC

English ▾

Q Search...

⌘K

Research

Login

Support

Discord

Sign up >

⚙

Welcome

Developer Platform

Claude Code

Model Context Protocol (MCP)

API Reference

Resources

Release Notes

Build with Claude

Learn how to get started with the Anthropic API, the Console, and Claude Code.

Ask Claude about docs...

Developer Platform



Get started

Make your first API call in minutes.



Features overview

Explore the advanced features and capabilities now available in Claude.



API reference

Integrate and scale using our API and SDKs.







Ask AI AI

입문자 도구 조합 비법

- 핵심 IDE: Cursor
- UI/웹 생성: v0, Lovable
- 온라인 협업·학습: Replit
- CLI/에이전트형 개발: Claude Code, Gemini Code
- 구현 아이디어만 있다면, 해커톤 현장에서 5대 도구 조합으로 30분, 1시간, 3시간만에 프로토타입 개선하고 배포

VibeCoding Master

모범적인 바이브코딩 방식

(Best Practices for VibeCoding)



이미 검증된 몇 가지 바이브코딩 패턴

- “혼자·빠르게·반복 배포”로 수요 검증 후 자동화
- AI는 전문성 재배치: 문맥·평가·전환 역량 강화
- 맹목 신뢰 금지, 반복 검증으로 신뢰 형성
- 개인 기여자(Individual Contributor)로서 창의·재미 우선

선배 바ای브코더: 피터 레벨스의 패턴

- 자금 조달 없이 마이크로/미니 스타트업 반복 출시
- 속도·실행력 극대화, 관료주의 우회
- 니치 문제 → 간단한 스택으로 MVP
- 수요 확인 후 자동화/최적화 단계 진입
- Nomad List, Remote OK, Photo AI

아이디어 발굴1: 문제 포착

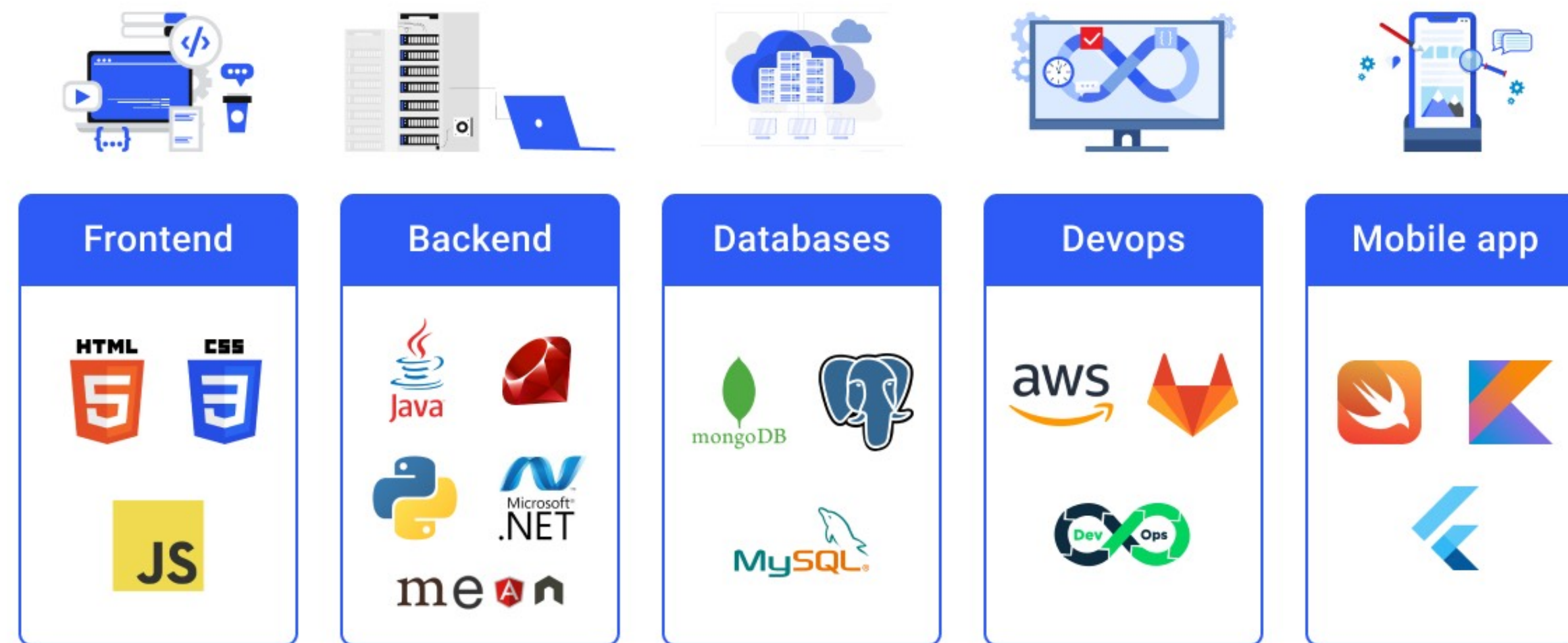
- 일상 불편을 목록화(Inbox·링크·정리 등)
- 자신에게 메시지(텔레그램/노트 앱)로 즉시 기록
- Trello/스프레드시트 등 가벼운 도구로 정리
- 잠재의식 숙성 → 타이밍 왔을 때 빠르게 착수

아이디어 발굴2: 기술/지리적 차이

- 기술 트렌드에서 가능성 탐색(Stable Diffusion, GPT)
- 다른 지역의 성공 모델을 로컬 문제에 적용
- 예: Photo AI—생성형 이미지의 실사용 가치 발굴
- 작은 실험으로 리스크 저비용 검증

기술 기반 선택: 웹앱, 모바일앱

Web Development Tech Stack



academysmart.com

iOS app development

Programming languages:



Objective-C



Swift

Toolkit:



Apple Xcode

SDK:



iOS SDK

Android app development

Programming languages:



Java



Kotlin

Toolkit:



Android Studio
& Android
Developer Tools

SDK:



Android SDK

Cross-platform app development

Programming languages:



React Native



Flutter

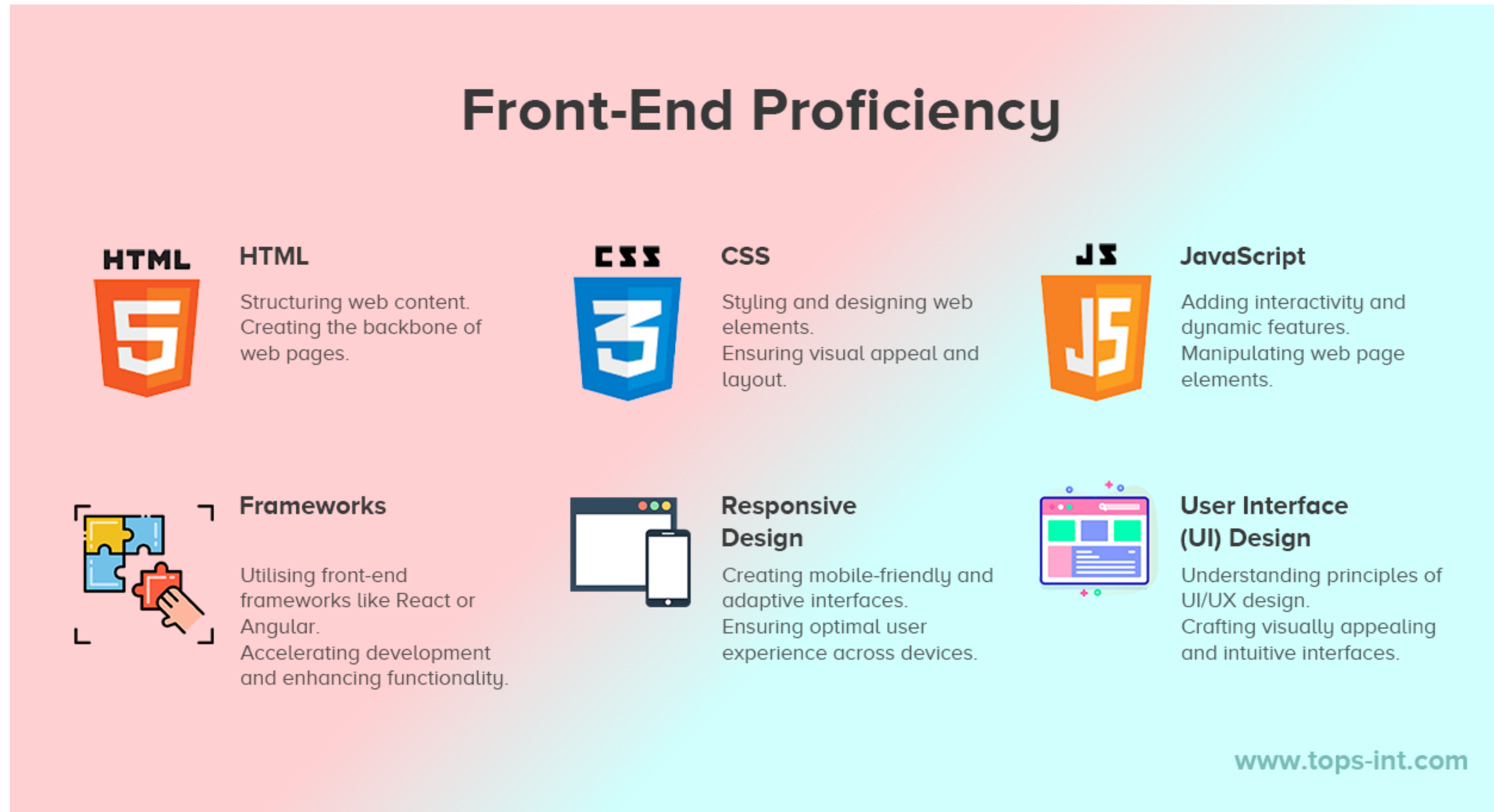


Unity

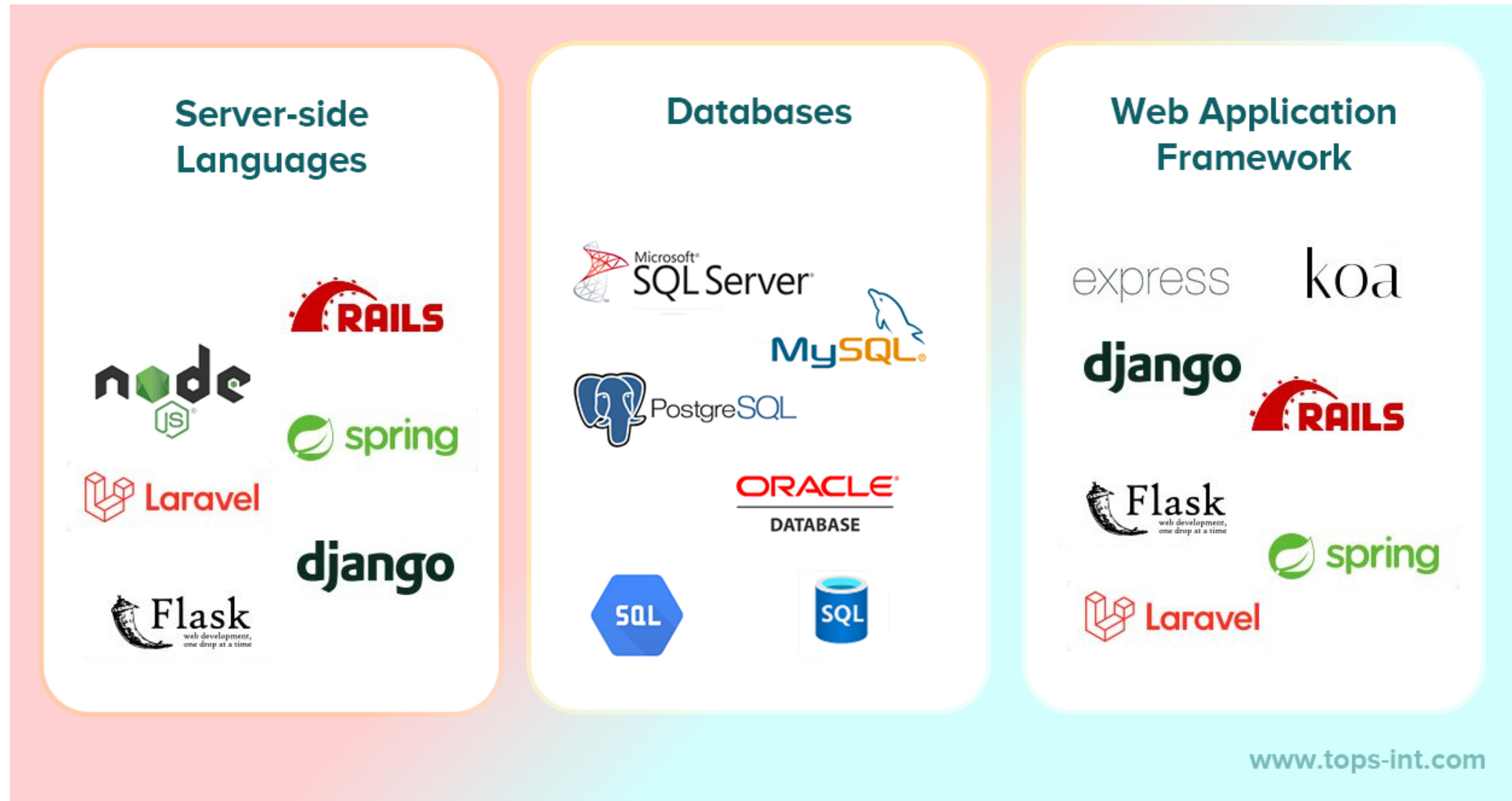


academysmart.com

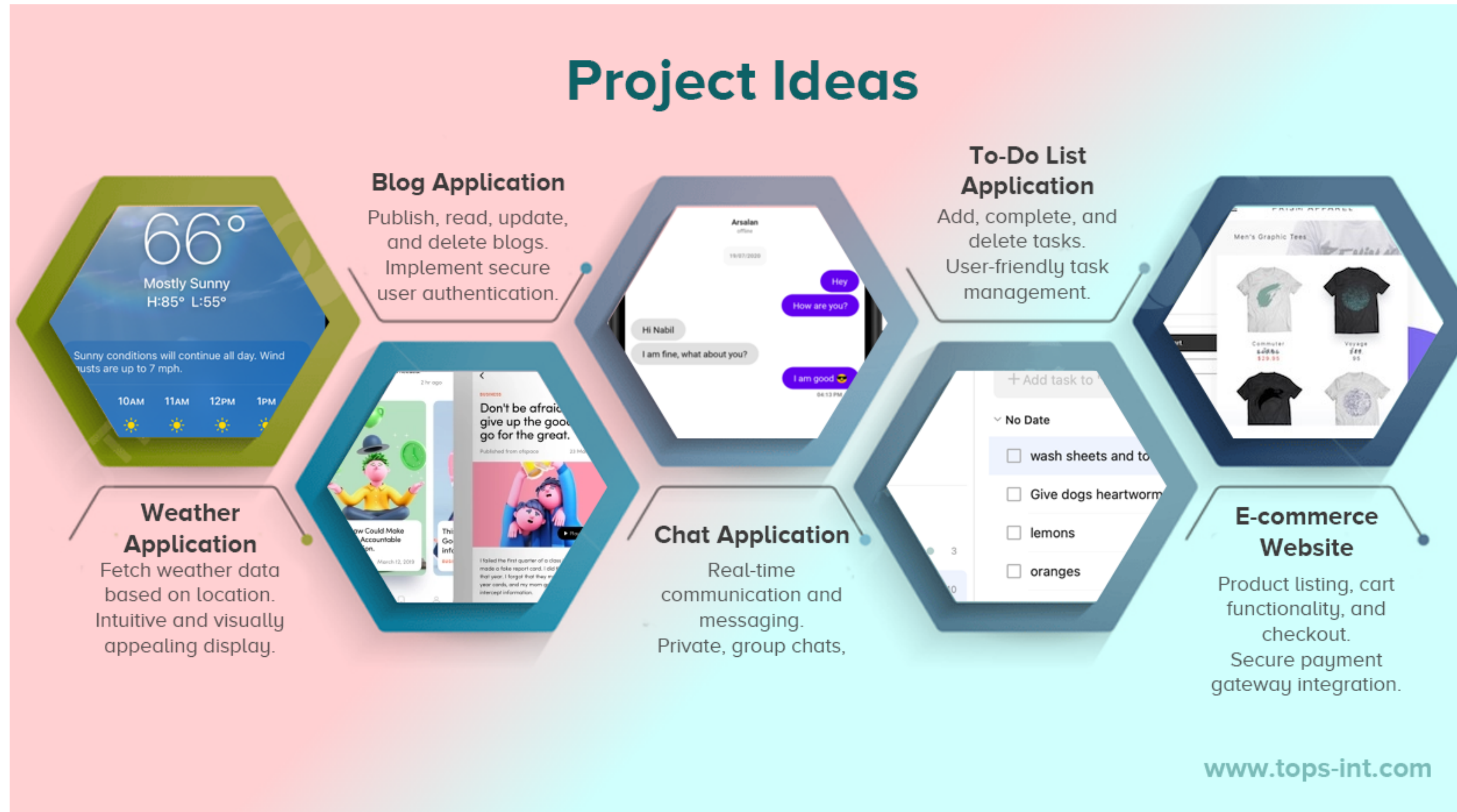
기술 기반 선택: 프론트엔드



기술 기반 선택: 백엔드 (서버&DB)



프로젝트 유형별 기술 기반



빌드 전략2: 내 주변 데이터부터 수집

- 초반엔 수동 처리로 수요 검증
- 유저 흐름 관찰 후 자동화 타이밍 판단
- 스프레드시트로 크라우드소싱(데이터/피드백)
- 예: Nomad List 초기 구글 시트 데이터 수집

신제품 출시 전략: 2주 내 출시

- 2주 이내 마이크로 제품 공개 목표
- 결제 의사(카드 입력)로 시장성 확인
- Building in Public: X/블로그로 투명 공유
- Shorts/TikTok/Reddit/HN로 유기 확산

성장 전략1: 피드백 루프 활용

- 부정적 피드백 포함, 모든 피드백에서 핵심 추출
- 포럼·디스코드로 커뮤니티 제안/토론
- 실사용 데이터로 우선순위 재조정
- 작은 개선의 빠른 릴리스 반복

VibeCoding Master

바이트코딩의 한계영역

(Gray Area of VibeCoding)



바이트코딩의 4대 한계 유형

- 소프트웨어 엔지니어링 유형별 한계
- 프로그래밍 언어 관점에서의 한계
- 프레임워크/라이브러리 관점에서의 한계
- 업무맥락적/조직문화적 제약 조건

SW: 미션 크리티컬 시스템



- 항공·철도 관제, 군사 시스템, 금융 코어 बैं킹, 원자력 제어
- 실시간성·안전성·법규 준수가 최우선, AI 코드 생성의 “불확실성”을 허용하기 어려움
- 코드 1줄 오류가 막대한 사고로 이어질 수 있어 “실험적·물입형 개발”보다 형식 검증(Formal Verification), 인증 프로세스가 요구됨

SW: 고성능/임베디드 시스템



- 반도체 펌웨어, 의료기기 내장 SW, 차량 ECU 등
- 리소스 제약(메모리, 전력) + 실시간 스케줄링 요구 → AI 자동생성 코드가 최적화된 C/어셈블리 수준을 보장하기
힘듦

SW: 대규모 분산/레거시 통합



- 20년 이상 된 레거시 시스템(COBOL, PL/I 등)과 연계
- AI가 코드를 제안해도 레거시 환경에 배포/테스트하기 어려워 **바이브코딩 루프(프롬프트→코드→실행)**가 깨짐

SW: 보안·규제 필수 영역



- 전자정부, 국방, 공공데이터 보안망
- 오픈AI API나 외부 IDE에 연결 불가 → 로컬 LLM + 온프레미스 IDE 도입 없이는 활용 곤란

프로그래밍 언어 관점

- 어셈블리, VHDL(HW 설계), Fortran, COBOL: 언어 자체가 저수준·도메인 특화, 실습·실행 환경이 일반 PC와 다름 
- 특수 목적 DSL(Domain-Specific Language): SAP ABAP, Oracle Forms 등 → 학습 데이터 희소성, 폐쇄적 생태계 
- Python, JavaScript, TypeScript, Go는 바이트코딩에 매우 적합 (풍부한 예제, 즉시 실행 환경, 빠른 배포 가능) 

프레임워크/라이브러리 관점



- 임베디드/실시간 OS용 프레임워크: FreeRTOS, QNX → 타이밍 제약
- EDA/시뮬레이션 라이브러리: Synopsys, Cadence → 상용, 폐쇄적, 복잡한 라이선스
- 고성능 HPC 프레임워크: MPI, OpenMP → 분산 환경 세팅 필요, 클라우드 실습 불가 시 어려움

프레임워크/라이브러리 관점



- 웹/앱 프레임워크: React, Next.js, SvelteKit, Flask, Django
- 자동화/데이터 파이프라인: LangChain, Airflow, Supabase, Firebase
- 크롬 확장·노코드 API 연동: Vercel, Netlify, Zapier

맥락적/문화적 제약



- 대기업/공공기관 망분리: API 호출 불가 → 바이트코딩 도입 어려움
- 개발 문화: “실험적 접근”보다 문서화·검증·심사 프로세스 우선
- 책임 구조: 오류 발생 시 책임 소재가 모호 → AI 코드 제안 신뢰도 낮음

결론: 직접적 소비자 니즈 충족



- 빠른 실험과 창의적 제품
- 다양한 의견 수렴 빌드
- 개인 창작자, 스타트업, 학습자 중심 폭발적 성장



- 안정·고신뢰·고규제 환경
- 대기업, 산업 제어, 미션 크리티컬, 보안 규제, 레거시 통합 영역

goorm



AI Fit-T 프로젝트 : 바이트코딩 (25.09.05~10.02)

Cursor, Code, v0 바이트코딩 20제 마스터